



Exercício: Sistema de Casa Inteligente (Smart Home)

Contexto

Você foi contratado para desenvolver o núcleo de um sistema de Casa Inteligente. O sistema precisa gerenciar diferentes tipos de dispositivos em um cômodo. Como você quer um código moderno, rápido e seguro em relação à memória (sem alocações dinâmicas complexas), você decidiu usar **POO clássica para os dispositivos** e `std::variant` **para armazená-los juntos**.

Requisitos do Sistema

1. Crie as Classes dos Dispositivos (POO)

Crie três classes independentes (elas **não** devem herdar de nenhuma classe base):

- **Lampada**
 - Atributos privados: `bool estado_ligada`, `int brilho` (0 a 100).
 - Métodos públicos: Construtor padrão, `ligar()`, `desligar()`, `setBrilho(int)`, e `imprimirStatus() const`.
- **Termostato**
 - Atributo privado: `float temperatura_atual`.
 - Métodos públicos: Construtor padrão, `setTemperatura(float)`, e `imprimirStatus() const`.
- **SensorPorta**
 - Atributo privado: `bool porta_aberta`.
 - Métodos públicos: Construtor padrão, `abrir()`, `fechar()`, e `imprimirStatus() const`.

2. Defina o Variant

Crie um *type alias* (apelido) usando `using` para representar um dispositivo genérico que pode ser qualquer um dos três acima:

```
#include <variant>
using Dispositivo = std::variant<Lampada, Termostato, SensorPorta>;
```

3. Crie a Classe Gerenciadora

Crie uma classe chamada `CasaInteligente`:

- **Atributo:** Um `std::vector<Dispositivo>` para guardar todos os aparelhos da casa.
- **Método** `adicionarDispositivo(const Dispositivo& d)`: Adiciona um dispositivo ao vetor.
- **Método** `ativarModoInverno()` (**Ação com `get_if`**):
 - Este método deve percorrer o vetor de dispositivos.
 - Usando `std::get_if`, verifique se o dispositivo atual é um `Termostato`.

- Se for, altere a temperatura dele para `24.5` graus. (*Dica: lembre-se que `get_if` trabalha com ponteiros!*).
- **Método `gerarRelatorio()` (Desafio):**
 - Percorra o vetor e chame o método `imprimirStatus()` de cada dispositivo.
 - *Opção A (Mais verbosa):* Use múltiplos `std::get_if` para checar qual é o tipo ativo e chamar o método.
 - *Opção B (Elegante/Avançada):* Pesquise e tente usar o `std::visit` com um *lambda genérico* (ou um functor) para chamar o método de forma automática, não importando qual tipo esteja no `variant`.

4. Função `main()`

Escreva a função principal para testar seu código:

1. Instancie uma `CasaInteligente`.
2. Crie uma `Lampada` (ligue-a e ponha brilho 80), um `Termostato` (temperatura 18.0) e um `SensorPorta` (fechada).
3. Adicione os três na `CasaInteligente`.
4. Chame `gerarRelatorio()`.
5. Chame `ativarModoInverno()`.
6. Chame `gerarRelatorio()` novamente para provar que apenas o termostato foi alterado.

Dicas para o Desenvolvimento

- Lembre-se de incluir as bibliotecas `<iostream>`, `<variant>`, e `<vector>`.
- O `std::get_if<Tipo>(&meu_variant)` retorna um ponteiro nulo (`nullptr`) se o `meu_variant` não estiver guardando um objeto do `Tipo`. Use um `if` simples para testar isso!